

PROJECT AND TEAM INFORMATION

Project Title

NodeSync : Fault-Tolerant Distributed Consensus Simulator For E-Commerce

Student / Team Information

<i>Team Name:</i> <i>Team # DSCPP-III-2026-T295</i>	<i>Beyond Binary</i>
Team member 1 (Team Lead)	<i>Aakshi –</i> <i>2510380120</i> <i>2510380120@geu.ac.in</i> 
Team member 2	<i>Tripathi, Astik –</i> <i>2510370444</i> <i>2510370444@geu.ac.in</i> 
Team member 3	<i>Sidharth –</i> <i>2512510032</i> <i>2512510032@geu.ac.in</i> 
Team member 4	<i>Mittal, Sanjam –</i> <i>2510370433</i> <i>2510370433@geu.ac.in</i> 

PROPOSAL DESCRIPTION

Motivation

Modern e-commerce applications are typically built using multiple backend servers to ensure scalability and reliability. However, such servers need to negotiate and agree on the order of potentially conflicting operations, such as updates to the same inventory or order details. In particular, if two or more backend servers store the same information and receive concurrent updates, the information on them may become out of sync. For instance, consider an order that requires more items than are in stock: such an order should be rejected, but if two servers receive the request concurrently, they may accept it, leading to an oversupply.

The situation becomes more complicated when the servers are unreliable or experience communication delays, as the system needs to be able to recover from failures gracefully.

The purpose of our project is to design and implement a C++ based distributed e-commerce simulator using the Raft consensus algorithm. The simulator will be used to demonstrate the basic principles of leader election, heartbeats, log replication, and majority voting. We will implement the mechanisms to handle leader failures and node failures, as well as network delays.

The project is important because it provides a practical way to understand how distributed systems maintain consistency and fault tolerance. It also allows us to apply DSA and C++ concepts such as graphs, queues, priority queues, hash maps, vectors and OOP to a real-world e-commerce scenario.

State of the Art / Current solution

In modern e-commerce systems, order and inventory consistency is usually implemented using centralized or distributed Order Management Systems (OMS), transactional databases, atomic inventory operations, distributed locking and event-driven architectures. Platforms like Shopify offer multi-location inventory management and order routing capabilities, while cloud-based OMS architectures use databases, APIs, event streams and workflow services to coordinate multiple order locations.

For concurrent inventory updates, systems usually make use of techniques like atomic database transactions, optimistic locking, reservations and distributed locks in order to prevent two customers from claiming the same stock. Modern distributed applications use event-driven patterns like Kafka and Saga-based workflows in order to coordinate order, payment and inventory services.

At the distributed-systems level, consensus protocols like Raft and Paxos are used in order to reach consensus or ordering between replicated servers. Raft uses a leader and followers, with operations replicated and committed after achieving a majority.

Our project simplifies these production approaches into a C++ simulator so that the underlying consensus and fault-tolerance mechanisms can be implemented, visualized and experimentally evaluated.

Project Goals and Milestones

The main idea is to build a simulator in C++ that will demonstrate how distributed nodes reach consensus and recover from failures, applying the concepts of DSA and C++ practically.

Project Goals:

- To design a virtual network and implement Raft for leader election, message exchange and log replication.*
- Simulate node failures, message delays and network partitions, and display the resulting node states and message flow.*
- Measure and compare election time, consensus time, recovery time and message count under different conditions.*

Initial Milestones:

- 1. Weeks 1–2: Study Raft, define requirements and design the node/network structure using graphs.*
- 2. Weeks 3–5: Implement message handling, leader election, heartbeats and log replication.*
- 3. Weeks 6–7: Add failure scenarios, network delays, partitions, logging, performance measurements and testing.*
- 4. Final Phase: Complete the interface, analyze results and prepare the final documentation and demonstration.*

Project Approach

Our solution is to implement a C++ based simulator for a distributed e-commerce backend, where multiple virtual-servers act as nodes and Raft consensus algorithm coordinates order and inventory operations while maintaining a consistent state.

The implementation will be composed of the following layers:

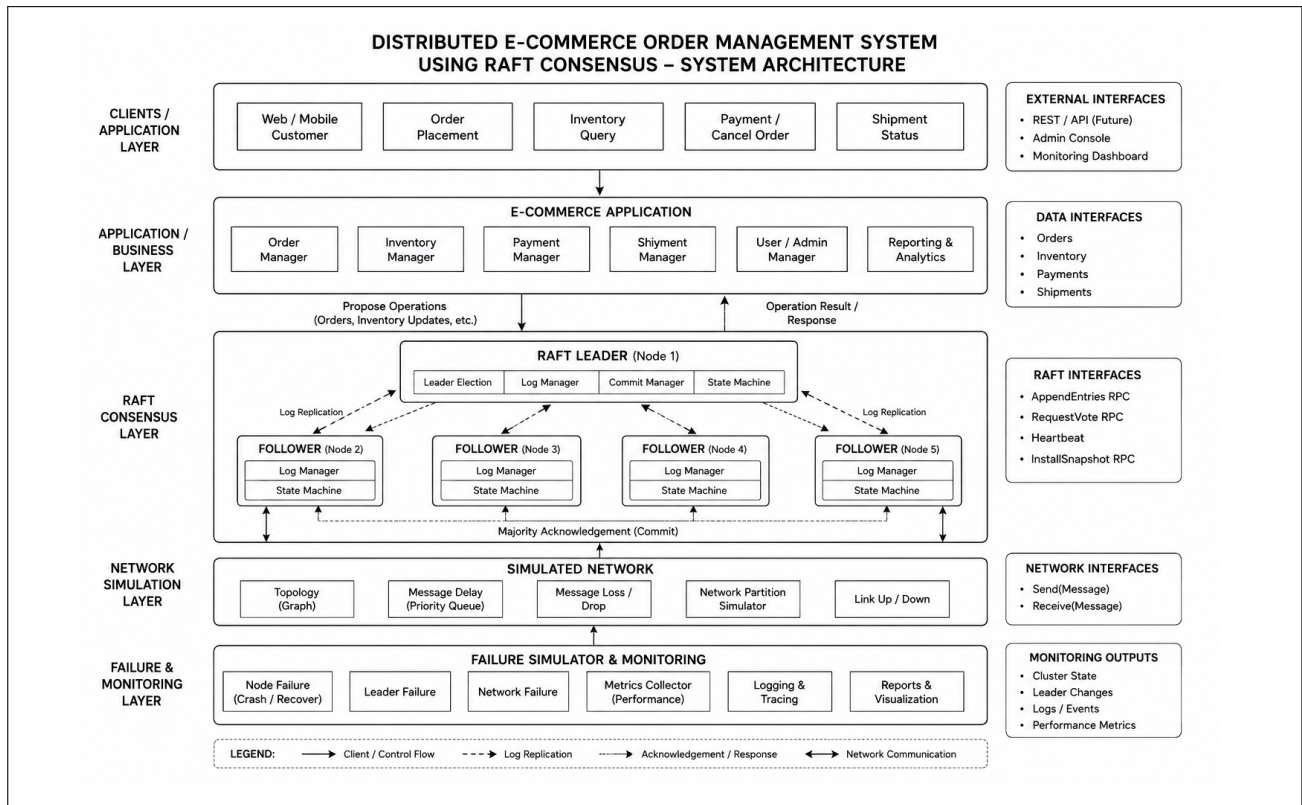
Distributed Node and Network: Nodes will be represented as C++ objects and organized in a graph/adjacency-list manner, where messages are treated as queues and priority queues are used to emulate different delivery-time priorities.

Raft and E-Commerce: A Raft engine will be used for leader election, heartbeats, log replication and majority-based commit and an order and inventory module will process create, reserve, cancel, and update-order operations.

Failure Simulation and Evaluation: The system will be subjected to various failure scenarios, such as leader or node failure, message loss, delays, or partitions. Appropriate logs such as election-time, consensus-latency, message-count, and recovery-time will be recorded and evaluated for comparative analysis.

The project will be implemented using C++17/C++20 and the Standard Template Library (STL), built with CMake and version-controlled with Git/GitHub. We propose to initially use a console-based interface to display node, leader or node failure, message, order and inventory status, and CSV/text output for log recording, with graphical visualization considered as a future enhancement.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The project will produce a working C++ based distributed e-commerce order management simulator that will demonstrate how multiple virtual backend nodes maintain consistent order and inventory information. The simulator will support basic order processing and inventory reservation while allowing different network and failure conditions to be tested.

The deliverables will be:

- A functional C++ simulator with multiple virtual nodes, distributed order and inventory management, and configurable network conditions.
- Failure simulation, event logging and performance metrics such as leader election time, consensus time, message count and recovery time.
- Test results comparing normal operation with different node failures, message delays, message loss and network partition scenarios.
- Complete C++ source code and documentation demonstrating OOP, STL and DSA concepts such as graphs, queues, priority queues, vectors and hash maps.

The final outcome will demonstrate the practical working of distributed consensus in an e-commerce scenario while showing how Data Structures and C++ programming concepts can be applied to a real-world problem.

Assumptions

- *The simulator will make use of a small set of virtual backend nodes, usually three to seven nodes, where each node maintains a copy of the required order and inventory information.*
- *Nodes can experience crash failures and/or temporary disconnections. Malicious, Byzantine, and security-related attacks are out of the initial project scope.*
- *Communication across the network will be simulated within the system, with controllable message delays, message loss, and temporary network partitions between nodes.*
- *The simulation will focus on basic e-commerce operations such as order processing and inventory reservation, as well as observing system behaviour during node failures and network disruptions.*

References

1. *Ongaro, D. and Ousterhout, J. — In Search of an Understandable Consensus Algorithm (Raft), Stanford University.
Raft Research Paper*
2. *Raft Consensus Algorithm — Official Raft website and research resources.
Raft Official Website*
3. *etcd Documentation — Raft-based distributed key-value store and consensus implementation.
etcd Documentation*
4. *Kubernetes Documentation — Operating etcd clusters for Kubernetes. Used to understand the real-world use of etcd in distributed systems. (Kubernetes)*
5. *cppreference — C++ Standard Library containers, including vector, queue, priority_queue, and unordered_map
(en.cppreference.com)*
6. *etcd Raft Package Documentation — Practical reference for a production Raft implementation.
(go.etcd.io)*